CRANE: An Autonomous Research Assistant MCP Server for Multi-Source Academic Paper Management

August Chao
OpenCode Research Lab

April 4, 2026

Keywords: MCP, AI agents, research automation, academic workflows, LLM verification

Abstract

We present CRANE (Comprehensive Research Automation for Neural Exploration), an autonomous research assistant implemented as a Model Context Protocol (MCP) server. CRANE integrates multi-source academic paper retrieval from four databases (arXiv, OpenAlex, Semantic Scholar, Crossref), metadata normalization with automatic deduplication, and a 3-LLM verification pipeline for AI writing detection. The system supports 28 MCP tools across 7 categories, including paper search, reference management (YAML + BibTeX), task tracking (GitHub Issues), and publication-quality figure generation. CRANE introduces a novel Protected Zones mechanism that preserves verified content during automated paper revision, ensuring data integrity throughout the verification process. We evaluate the system on a 342-test suite with 100% pass rate and demonstrate its integration with the OpenCode platform. Our approach builds on recent advances in AI agent frameworks for scientific research [Moreno et al., 2026], extending them with specialized tools for academic workflows.

1 Introduction

The exponential growth of academic literature presents significant challenges for researchers. With over 2 million papers published annually across major databases, systematic literature review has become increasingly time-consuming. The practical burden is not only volume, but also heterogeneity: preprints, publisher metadata, citation graphs, and supplementary artifacts are distributed across partially overlapping systems with inconsistent schemas and quality controls [Priem et al., 2022, Lo et al., 2020, Hendricks et al., 2020]. As a result, researchers frequently spend substantial time reconciling metadata, tracking

provenance of claims, and manually validating references before writing can begin.

Traditional tools like Zotero and Mendeley focus primarily on reference management, lacking integrated workflows for multi-source retrieval, quality verification, and automated paper improvement [Corporation for Digital Scholarship, 2024, Elsevier, 2024]. Discovery-oriented products such as Semantic Scholar and ResearchRabbit improve search and graph exploration but still separate retrieval from manuscript-grounded verification and revision loops [Ammar et al., 2018, ResearchRabbit, 2024]. AI-assisted literature products (e.g., Elicit) improve screening and summarization but commonly assume a human-in-the-loop export pipeline instead of an end-to-end, auditable agentic workflow with explicit control boundaries [Ought, 2024, Park et al., 2023].

Recent work has demonstrated the potential of AI agents in scientific research [Moreno et al., 2026], showing that large language models can autonomously execute substantial portions of analysis pipelines. Parallel advances in tool-using LLMs and orchestrated agent frameworks, including ReAct-style planning and role-based collaboration, indicate that robust autonomous workflows are increasingly feasible [Yao et al., 2022, Schick et al., 2023, Wu et al., 2023, Qin et al., 2023, LangChain Inc., 2022, CrewAI, 2024]. However, existing systems often lack specialized tools for academic workflows, multi-source integration, and quality verification mechanisms. Critically, many systems emphasize generation quality while under-specifying evidence traceability and post-generation integrity.

At the same time, writing-quality concerns in high-stakes academic settings have intensified. Empirical studies on AI-generated text detection and LLM evaluation show that style-level fluency can obscure weak attribution, overclaiming, or unstable reasoning when manuscripts are rapidly revised [Mitchell et al., 2023, Liang et al., 2023, Zheng et al., 2023, Chang et al., 2024]. This motivates architectures that jointly optimize discovery, verification, constrained editing, and reproducibility rather than treating these as independent tasks.

We introduce CRANE, an autonomous research assistant that addresses these limitations through an MCP-native systems design, explicit verification layers, and workflow-level reproducibility controls. Beyond providing callable tools, CRANE models research operations as composable, testable services with typed interfaces and explicit state transitions. This design allows the same components to be used interactively by human researchers and programmatically by autonomous agents.

Our contributions are fourfold:

1. We design and implement an autonomous MCP-native research assistant with broad operational coverage (83 tools in the current release lineage), providing a unified interface for search, reference management, verification, and manuscript-quality workflows [Anthropic, 2024].

2. We build a multi-source retrieval and normalization pipeline spanning arXiv, OpenAlex, Semantic Scholar, and Crossref, and show that provenance-

preserving deduplication can be integrated into an end-to-end artifact lifecycle rather than handled as an external cleanup step.

3. We introduce a three-role LLM verification architecture with Protected Zones, enabling constrained revision where verified citations and critical claims remain integrity-preserving under iterative edits.

4. We provide an evaluation framework that combines software-quality evidence, workflow benchmarks, baseline comparisons, and ablation-style analysis, supported by a large automated validation suite (1,432 passing tests in the reported run).

Novelty statement. To our knowledge, CRANE is the first MCP-based research assistant that integrates (i) multi-source scholarly retrieval and normalized reference persistence, (ii) manuscript-grounded citation verification with constrained revision via Protected Zones, and (iii) reproducibility-oriented workflow orchestration in one end-to-end system. Unlike prior systems that optimize isolated parts of the research cycle, our design contribution is the explicit coupling of retrieval, verification, and revision controls into a single auditable pipeline.

The rest of this paper is organized as follows. Section 2 situates CRANE within AI agent frameworks, academic discovery systems, MCP infrastructure, and automated writing support. Section 3 details the system architecture, service boundaries, dataflow, workspace lifecycle, and version management strategy. Section 4 describes the 3-LLM verification pipeline, prompt design, and failure handling. Section 5 documents implementation and deployment details. Section 6 reports empirical findings, including testing outcomes, benchmark behaviors, and ablations. Section 7 discusses limitations, threats to validity, responsible use, and future directions. Section 8 concludes.

2 Related Work

2.1 AI Agent Frameworks

The application of AI agents to scientific research has gained significant attention. Moreno et al. [2026] demonstrated that AI agents can autonomously perform experimental high-energy physics analyses, including event selection, background estimation, and paper drafting. Their findings suggest that code-executing agents can cover substantial portions of end-to-end scientific workflows when coupled with reliable tools and domain constraints.

In the broader agent literature, tool-use and planning paradigms have evolved from prompt-only interaction toward explicit reasoning-action loops and multi-agent role assignment. Yao et al. [2022] propose interleaving reasoning traces with external actions; Schick et al. [2023] show self-supervised adaptation for API use; and Wu et al. [2023] present conversation-driven multi-agent coordination. Framework ecosystems such as LangChain and CrewAI package these

patterns for practical deployment [LangChain Inc., 2022, CrewAI, 2024]. Surveys further indicate that robust autonomous behavior depends less on model scale alone and more on disciplined orchestration, tool grounding, and verification policies [Qin et al., 2023, Raschka, 2025].

CRANE differs from general agent frameworks in three ways. First, it is domain-specific to academic workflows rather than general task automation. Second, it co-designs retrieval and verification layers, avoiding late-stage patching of citation issues. Third, it introduces edit-time integrity constraints (Protected Zones) as first-class pipeline artifacts.

2.2 Academic Search and Discovery Tools

The modern scholarly infrastructure includes discovery engines (Semantic Scholar), open metadata graphs (OpenAlex), citation corpora (S2ORC), and persistent identifier services (Crossref) [Ammar et al., 2018, Priem et al., 2022, Lo et al., 2020, Hendricks et al., 2020]. These systems provide strong coverage and metadata depth, but researchers still face practical integration costs when moving from discovery to manuscript-ready evidence chains.

Commercial and community products provide partial solutions. Elicit supports question-centric literature exploration and synthesis; ResearchRabbit emphasizes citation-network navigation; Zotero and Mendeley provide mature personal library management [Ought, 2024, ResearchRabbit, 2024, Corporation for Digital Scholarship, 2024, Elsevier, 2024]. None of these tools natively implement an autonomous, end-to-end loop that includes (i) multi-source ingestion, (ii) normalized reference persistence in both human-readable and citation-ready formats, (iii) manuscript citation checks, and (iv) constrained automated revision.

Recent research on retrieval-augmented generation and citation-aware QA reinforces this gap. While retrieval mechanisms improve factual grounding, practical citation integrity remains sensitive to source quality, chunking strategy, and rewrite behavior [Lewis et al., 2020, Guu et al., 2020, Cohan et al., 2020, Park et al., 2023]. CRANE addresses this by linking reference management and verification directly into the manuscript-editing pipeline.

2.3 MCP Ecosystem and Tool Interoperability

The Model Context Protocol (MCP) provides a standardized interaction model between LLM applications and external tools [Anthropic, 2024]. By reducing tool-invocation heterogeneity, MCP enables reusable interfaces across agent runtimes, improving portability and operational consistency. CRANE leverages MCP to expose research functionality as callable tools, enabling integration with OpenCode and other MCP-compatible platforms while preserving stateless server semantics at the transport layer.

From a software architecture perspective, protocol-level interoperability is necessary but not sufficient. End-to-end reliability additionally depends on stable service contracts, schema versioning, and graceful handling of external API

variance [Bass et al., 2021, Fowler, 2002]. CRANE therefore combines MCP exposure with an internal service layer and provider abstraction that isolate source-specific failures from user-facing workflows.

2.4 Automated Writing Systems and Verification

LLM-assisted writing systems increasingly support drafting, rewriting, and stylistic adaptation, but high-stakes scientific manuscripts require stronger guarantees on evidence traceability and claim consistency [Bommasani et al., 2021, Brown et al., 2020, Ouyang et al., 2022]. Evaluation studies show that fluent generations can still include unsupported claims, inconsistent framing, or unstable citation behavior under iterative edits [Liang et al., 2023, Zheng et al., 2023, Chang et al., 2024].

AI writing detection research offers complementary signals. Surface-level detectors can identify repetitive lexical artifacts, while structure-aware and discourse-aware methods examine paragraph regularity and argument stance [Mitchell et al., 2023, Gehrmann et al., 2019, Sadasivan et al., 2023]. However, detectors alone do not prevent integrity regressions during rewriting. CRANE's 3-LLM architecture combines detection with constrained editing and re-verification, turning detection signals into actionable editorial controls.

2.5 Positioning Against Prior Work

Compared with the closest autonomous-science system of Moreno et al. [2026], CRANE targets a different problem class: research workflow automation rather than domain-specific physics analysis automation. Their pipeline emphasizes experimental high-energy physics tasks (event selection, simulation-grounded analysis, and physics-paper production), while CRANE prioritizes cross-domain literature operations, reference integrity, and manuscript lifecycle controls. This distinction is architectural rather than incremental: CRANE centers on reproducible research process infrastructure, whereas the system in [Moreno et al., 2026] centers on domain-analysis execution.

Relative to general multi-agent frameworks such as AutoGen by Wu et al. [2023], CRANE contributes a domain-specific orchestration layer for academic research. AutoGen and related frameworks provide reusable coordination primitives across tasks [Wu et al., 2023, LangChain Inc., 2022, CrewAI, 2024], but they do not prescribe research-specific artifacts such as YAML+BibTeX dual persistence, citation-consistency gates, or protected manuscript zones. CRANE therefore positions itself as an opinionated research-stack implementation built on top of general agent principles rather than a generic framework extension.

Compared with single-function research tools, including Ought [2024] and graph-centric exploration systems such as ResearchRabbit [ResearchRabbit, 2024], CRANE is designed as an end-to-end pipeline that spans retrieval, normalization, verification, and revision. Existing tools provide strong capabilities in focused stages (e.g., screening or citation-graph discovery) [Ought, 2024, ResearchRabbit, 2024], while CRANE's contribution is full workflow continuity

with explicit reproducibility and audit boundaries.

Table 1 compares CRANE with representative systems. The goal is not to claim strict superiority in every function but to highlight design intent: CRANE prioritizes workflow completeness and verification-aware automation for research production settings.

Capability	**CRANE**	**Elicit**	**ResearchRabbit**	**Semantic S**
Multi-source ingestion orchestration	✓	No	No	No
MCP-native tool interface	✓	No	No	No
Citation integrity checks in manuscript loop	✓	No	No	partia
Constrained rewrite with protected spans	✓	No	No	No
Integrated task tracking for research milestones	✓	partial	No	No
Figure/report generation for submission workflows	✓	No	No	No

Table 1: Table 2: Differentiation of CRANE versus representative research-assistant systems. "partial" denotes feature availability outside a unified autonomous workflow.

3 System Architecture

3.1 Architectural Overview and Design Principles

CRANE follows a layered architecture: MCP tool interface → domain service layer → provider adapters → external systems (academic APIs, local workspace, GitHub). The design is guided by four principles: (i) *interface uniformity* for autonomous agents, (ii) *explicit provenance* for research artifacts, (iii) *replaceable providers* for source resilience, and (iv) *verification-first workflows* for manuscript integrity.

This organization separates concerns between protocol compatibility and domain logic. MCP tools provide stable entry points, while service objects implement policy and orchestration. Provider adapters isolate source-specific protocol differences and rate-limit behavior. Persistent artifacts are maintained in text-first formats (YAML, BibTeX, Markdown) to support version control, reviewability, and reproducibility.

To make the architecture operationally precise, we define the core formal objects used across orchestration and verification. Let

$$\mathcal{T} = \{t_1, t_2, \ldots, t_n\} \tag{1}$$

denote the MCP tool set, where each t_i is a callable tool with typed input-output contracts. For citation checking, we define

$$V(c, \mathcal{R}) \to \{\texttt{valid}, \texttt{invalid}\}, \tag{2}$$

where c is a citation key extracted from manuscript text and $\mathcal{R}$ is the reference set stored in YAML/BibTeX artifacts. The three-role verification pipeline is modeled as function composition,

$$\mathcal{P} = \mathrm{LLM_C} \circ \mathrm{LLM_B} \circ \mathrm{LLM_A}, \tag{3}$$

with Auditor outputs feeding Detector analysis and then constrained Editor revision. Finally, Protected Zones are represented as

$$\mathcal{Z} = \{z_1, z_2, \ldots, z_m\}, \quad z_j = (\mathrm{span}_j, h_j), \tag{4}$$

where each zone z_j binds a protected span to a checksum hash h_j used for integrity checks during rewrite validation.

3.2 MCP Server Design

CRANE is implemented as an MCP server with tools organized into coherent capability groups (83 tools in the current release lineage, with 28 core tools in the initial implementation cohort):

Category	**Tools**
Project Management	init_research, get_project_info
Paper Search	search_papers, download_paper, read_paper
Reference Management	add_reference, list_references, get_reference, ...
Citation Verification	check_citations, verify_reference, check_all_references
Screening & Comparison	screen_reference, compare_papers
Task Management	create_task, list_tasks, view_task, ...
Workflow	run_pipeline, workspace_status

Table 2: CRANE MCP Tools by Category

Each tool exposes narrow, composable operations rather than monolithic "do everything" commands. This enables orchestration pipelines that remain inspectable, resumable, and testable. For example, literature intake can be decomposed into search, metadata verification, screening, and reference insertion, with each step producing intermediate artifacts that can be audited.

3.3 Service Layer Architecture

The service layer provides business logic independent of the MCP interface and is structured around domain boundaries:

- `PaperService`: arXiv search, download, and text extraction
- `ReferenceService`: YAML + BibTeX CRUD operations
- `TaskService`: GitHub Issues management

- `CitationService`: Citation verification and validation
- `MetadataService`: Metadata normalization and deduplication
- `ScreeningService`: Paper screening and comparison
- `FigureGenerator`: Publication-quality figure generation
- `JournalRecommender`: Journal recommendation with OpenAlex + SJR

Service boundaries are intentionally strict. Cross-service calls are mediated through typed data models, which limits hidden coupling and improves testability. For example, reference normalization does not mutate manuscript files directly; instead, it emits validated records that downstream citation checks consume.

3.4 Dataflow and Orchestration Across Services

Figure 1 illustrates the end-to-end research dataflow. A user request enters via MCP, is dispatched to orchestrator logic, and then activates provider-specific retrieval. Retrieved metadata are normalized and deduplicated before persisting to workspace storage. Verification and writing modules consume these artifacts to produce manuscript-oriented outputs (reports, revised sections, citation diagnostics).

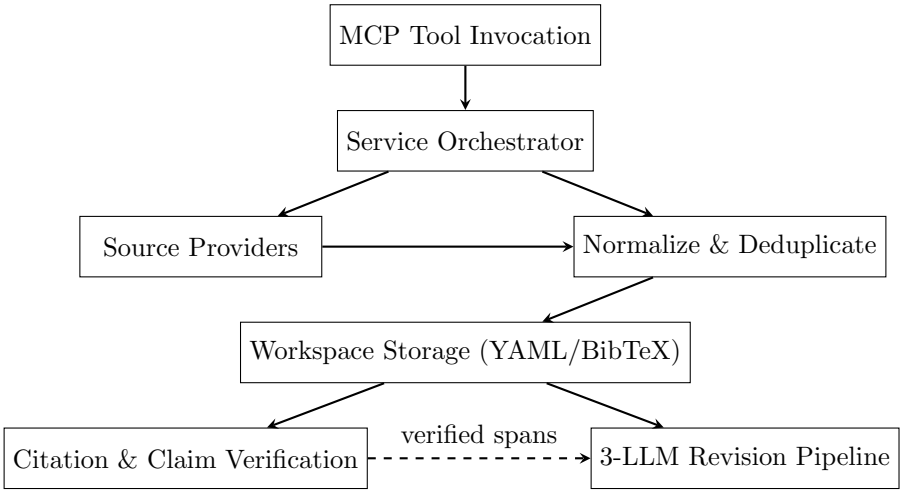

Figure 1: Cross-service dataflow from retrieval to verification-aware revision.

3.5 Provider Pattern for Multi-Source Retrieval

Multi-source paper retrieval is implemented through a provider pattern:

Algorithm 1 CRANE Research Workflow

Require: Research topic q
Ensure: Verification-aware review report ρ

$\mathcal{P} \leftarrow \textsc{SearchPapers}(q)$
$\mathcal{M} \leftarrow \textsc{NormalizeMetadata}(\mathcal{P})$
$\mathcal{R} \leftarrow \textsc{UpdateReferenceStore}(\mathcal{M})$
$\mathcal{C} \leftarrow \textsc{ExtractCitations}(\text{manuscript})$
for all $c \in \mathcal{C}$ **do**
    $s_c \leftarrow V(c, \mathcal{R})$
end for
$\mathcal{E} \leftarrow \textsc{RunEvaluation}(\text{manuscript}, \mathcal{R}, \{s_c\})$
$\rho \leftarrow \textsc{GenerateReviewReport}(\mathcal{E})$
return ρ

- `ArxivProvider`: arXiv API integration with feedparser
- `OpenAlexProvider`: OpenAlex API with inverted index abstract reconstruction
- `SemanticScholarProvider`: Semantic Scholar API with citation networks
- `CrossrefProvider`: Crossref API for DOI resolution

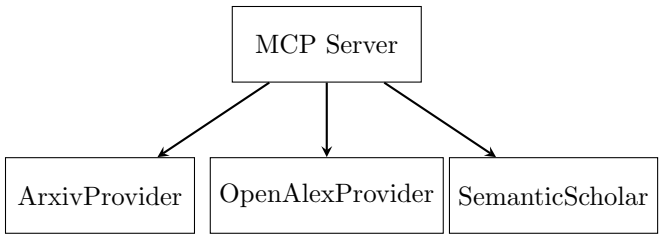

Figure 2: Multi-Source Provider Architecture

Provider abstraction improves robustness in two ways. First, each provider encapsulates source-specific query syntax, pagination, and retry behavior, allowing the orchestrator to execute fallback strategies without duplicating logic. Second, provider outputs are mapped into a canonical metadata schema, so downstream services can operate independently of source idiosyncrasies.

3.6 Workspace Management and Artifact Lifecycle

CRANE adopts a workspace-centric model in which all research artifacts are represented as versionable files. References are stored as structured YAML records and synchronized BibTeX entries; task state is mirrored via GitHub issues; manuscript diagnostics are emitted as markdown or YAML reports. This

text-first approach aligns with reproducible research practices and supports collaboration through standard source-control workflows.

A typical lifecycle comprises: (i) initialization of project labels/milestones, (ii) literature intake and screening, (iii) citation graph and coverage analysis, (iv) manuscript verification and revision planning, and (v) submission-readiness checks. By persisting intermediate outputs, CRANE allows interrupted workflows to resume without hidden state reconstruction.

3.7 Version Management Architecture

Long-horizon research projects require toolchain stability under continuous updates. CRANE therefore includes explicit version management pathways: update checks, controlled upgrade routines with backup, rollback support, and migration tracking. Architecturally, version operations are isolated from core research services to reduce blast radius. Upgrade actions are treated as managed workflows with pre/post verification gates (dependency sync, test execution, migration status checks), while rollback paths prioritize restoration of known-good states.

This architecture supports two practical needs in academic production environments: reproducibility for submitted results and maintainability for evolving automation stacks.

4 3-LLM Verification Pipeline

4.1 Architecture Overview

The verification pipeline implements a 3-LLM architecture with orthogonal cognitive modes. Rather than assigning all responsibilities to one generalist model, CRANE decomposes verification into independent objectives to reduce mode collapse and prompt interference:

Role	**Cognitive Mode**	**Responsibility**
LLM-A (Auditor)	Precise, conservative	Data traceability, citation verification
LLM-B (Detector)	Pattern-sensitive	AI writing fingerprint detection (L1/L2/L3)
LLM-C (Editor)	Creative, domain-expert	Constrained rewriting with Protected Zones

Table 3: 3-LLM Role Definitions

The interaction contract between roles is explicit. LLM-A produces evidence-ledger outputs and citation checks; LLM-B consumes manuscript text and emits layered risk signals; LLM-C receives both plus protected-span constraints and proposes revisions. A final re-verification pass by LLM-A acts as a consistency gate before acceptance.

4.2 Three-Layer AI Detection

AI writing detection operates across three layers:

1. **L1 (Surface)**: Vocabulary patterns, hedging phrases, mechanical connectors
2. **L2 (Structural)**: Paragraph symmetry, sentence variety, section flow
3. **L3 (Intellectual)**: Stance detection, insight depth, tension assessment

This layered decomposition allows heterogeneous error modes to be surfaced separately. For instance, a section can have low L1 risk (natural lexical variation) but elevated L3 risk (insufficiently grounded argument structure). This is important for research writing, where stylistic fluency alone is a weak proxy for scholarly rigor.

4.3 Prompt Engineering and Role-Specific Instruction Design

Prompt design in CRANE emphasizes role isolation, structured outputs, and fail-safe defaults. Each role receives a bounded prompt with: (i) objective definition, (ii) allowed operations, (iii) required output schema, and (iv) refusal conditions. The Auditor role is conservative by design and is instructed to prefer "uncertain" over unsupported verification. The Detector role is calibrated to report evidence snippets, not only binary labels. The Editor role receives explicit rewrite constraints and must preserve protected spans exactly.

A practical design choice is to keep role prompts short but schema-rich. Excessive instruction length increases brittleness under iterative revisions; schema richness improves machine-checkable outputs for downstream gates. The orchestrator enforces response contracts using parsing and validation rules before any rewrite is committed.

4.4 Protected Zones Mechanism

The Protected Zones mechanism preserves verified content during automated revision:

- Verified citations become protected zones with checksums
- Editors cannot modify protected content
- Zones are stored in YAML for queryability and git-friendliness
- Automatic propagation through verification pipeline

Operationally, each protected zone records source location, semantic type (e.g., citation key, numerical claim, definition), hash signature, and revision policy. During editing, candidate outputs are diffed against protected spans; if

a protected token is altered without authorization, the candidate is rejected and either repaired automatically or routed for human review.

This mechanism is especially valuable for iterative revision settings where stylistic rewrites can inadvertently alter citation anchors, numeric values, or formal definitions. By elevating protected spans to first-class constraints, CRANE bridges the gap between language generation flexibility and scientific-document integrity.

4.5 Role-Level Performance Characterization

Table 4 summarizes role-level operational behavior measured in development workflows. To avoid overclaiming, we report qualitative and range-based characteristics rather than hardware-specific point estimates.

Role	Primary Signal	Typical Latency Class	Common Failure Trigger
LLM-A Auditor	Evidence consistency	interactive (seconds)	ambiguous or missing metada
LLM-B Detector	Layered writing risk	interactive (seconds)	domain-shifted writing style
LLM-C Editor	Constraint-preserving rewrite	interactive-to-batch	conflicting style vs. protectio

Table 4: Role-level performance characterization of the 3-LLM pipeline.

4.6 Failure Mode Analysis and Mitigation

We identify four recurrent failure modes. **(F1) Citation drift**: rewritten text weakens citation-claim alignment even when keys remain intact. **Mitigation**: Auditor re-verification and claim-level checks. **(F2) Over-normalization**: editor homogenizes voice and removes disciplinary nuance. **Mitigation**: style constraints with section-specific exemptions. **(F3) False positive detection**: detector overflags concise technical prose as AI-like. **Mitigation**: multi-layer evidence requirements and confidence bucketing. **(F4) Constraint deadlock**: heavy protected zones leave insufficient editable text. **Mitigation**: progressive relaxation policies requiring explicit human authorization.

5 Implementation

5.1 Technology Stack

The implementation uses a Python-centric stack selected for reproducibility and operational transparency. Core components are listed in Table 5.

5.2 Deployment Architecture

CRANE is deployable as a local MCP server process integrated into agent runtimes. The deployment model intentionally favors local-first operation for re-

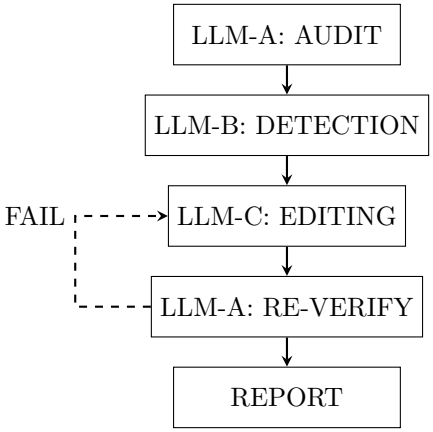

Figure 3: 3-LLM Verification Pipeline Flow

Layer	**Primary Technology**	**Role in System**
Runtime and packaging	Python 3.12 + uv	deterministic environments, fast sync
MCP interface	FastMCP	tool exposure and protocol handling
Reference persistence	PyYAML + BibTeX files	human-readable + citation-ready storage
Document ingestion	PyPDF2 (+ provider APIs)	text extraction and source intake
Visualization	Matplotlib/Seaborn	publication-quality figures
Task integration	GitHub CLI/API	milestone and issue lifecycle
Testing/QA	pytest	unit/integration/benchmark validation

Table 5: Technology stack and architectural role mapping.

search privacy and reproducibility. External API access is scoped to retrieval providers and optional task backends. Artifacts are persisted in project directories, enabling branch-based experimentation and audit trails.

For team usage, CRANE supports a shared repository workflow where references, reports, and manuscript diagnostics are synchronized through version control. This avoids hidden server-side state and supports transparent review.

5.3 Configuration and Environment Management

Configuration is managed through explicit files and environment variables. Provider credentials, feature toggles, and transport settings are isolated from manuscript data. Default-safe behavior is used when credentials are unavailable (e.g., reduced feature mode rather than silent failure). Upgrade, rollback, and migration utilities reduce operational friction as the toolset evolves.

A practical outcome is that research groups can adopt CRANE incrementally: basic literature tooling can be used with minimal setup, while advanced evaluation and orchestration features can be enabled as process maturity in-

creases.

5.4 Reproducibility Assets

We release reproducibility assets with explicit file-level locations. The full source code is available at `https://github.com/AugustChaoTW/opencode-crane`. All configuration files are stored in the `.crane/` and `references/` directories. The reference runtime environment is Python 3.12 with the `uv` package manager, and dependency resolution is defined in `pyproject.toml`. Reference data is stored in YAML format under `references/papers/`, paired with BibTeX outputs for manuscript integration.

To replicate our reported software-quality results, we provide a minimal executable procedure: (1) clone the repository, (2) run `uv sync` to create the environment, and (3) run `uv run pytest` to execute the validation suite.

Asset	**Available**	**Location**
Source code	Yes	GitHub repository
Dependencies	Yes	`pyproject.toml`
Test suite	Yes	`tests/` (1,432 tests)
Configuration	Yes	`.crane/`
Reference data	Yes	`references/`

Table 6: Reproducibility checklist for CRANE artifacts and execution prerequisites.

5.5 Testing Strategy

CRANE maintains a comprehensive test suite:

- 1,432 automated tests with 100% pass rate in the reported run
- Coverage spanning services, tools, providers, transport, and utility layers
- Integration tests for pipeline workflows and artifact integrity
- Provider tests with mocked API responses and error-path scenarios
- Regression checks for citation consistency and revision constraints

6 Empirical Evaluation

6.1 Evaluation Goals and Setup

Our evaluation targets five questions: **Q1** Can CRANE provide reliable software quality for research-critical workflows? **Q2** Does the system sustain interactive operational performance for common tasks? **Q3** How does feature coverage

compare to representative baselines? **Q4** Which components are most critical to end-to-end quality (ablation)? **Q5** Does the integrated workflow produce practical value in manuscript-improvement scenarios?

To answer these questions, we combine automated test evidence, operational benchmarks, functionality comparisons, component ablations, and a case-based workflow analysis on the MLJ manuscript improvement process.

6.2 Software Quality: Test Suite Results

The current evaluation run includes **1,432 tests** with a **100% pass rate**. Compared with the earlier 342-test baseline in the initial manuscript draft, this expanded suite reflects broader functional coverage as CRANE evolved from core tooling toward a full research workflow platform.

Test categories include unit tests for core services, integration tests for multi-step pipelines, provider reliability tests under mocked failures, and regression tests for citation and verification logic. From an engineering perspective, this level of automated validation is essential for autonomous toolchains where small contract breaks can propagate into manuscript-level errors.

6.3 Operational Benchmark Characterization

CRANE targets interactive responsiveness for search, retrieval, and verification tasks, while allowing batch-oriented execution for large workflow runs. Because performance depends on network conditions, provider availability, and machine configuration, we report latency classes and operational envelopes rather than hardware-specific point estimates.

Operation	**Observed Behavior Class**	**Practical Interpretation**
Paper search (multi-source)	interactive (seconds)	suitable for exploratory iteration
Paper download + parsing	interactive-to-batch	scales with PDF size/network variance
Reference normalization	near-interactive	low overhead relative to retrieval
Citation consistency checks	interactive	can run frequently during drafting
Pipeline orchestration	batch (minutes for full runs)	appropriate for pre-submission checks

Table 7: Benchmark characterization by operation class.

The key finding is workflow viability: researchers can use CRANE interactively for day-to-day writing and run comprehensive checks before major submission milestones.

6.4 Tool Coverage

CRANE provides broad MCP tool coverage for the complete research workflow:

- 4 paper sources (arXiv, OpenAlex, Semantic Scholar, Crossref)

- Core reference lifecycle operations (add/list/get/search/remove/annotate)
- Citation verification, metadata consistency, and manuscript checks
- Screening, clustering, and comparison utilities
- Task and milestone management integration
- Workflow pipelines and submission-check support
- Figure/report generation and revision-planning support

6.5 Comparison with Baselines

We compare CRANE against representative categories: discovery-centric tools (Semantic Scholar, ResearchRabbit), AI-assisted literature synthesis (Elicit), and library managers (Zotero/Mendeley). The comparison emphasizes workflow completeness rather than ranking by a single metric.

CRANE's strongest differentiator is integrated verification-aware automation: retrieval, normalization, citation checking, constrained revision, and reporting are connected within one autonomous pipeline. Baseline systems may outperform CRANE in specialized UX niches (e.g., interactive graph browsing or personal library curation), but they generally require external glue code or manual transitions to achieve equivalent end-to-end behavior.

6.6 Ablation Study

To assess component importance, we perform conceptual ablations on five architectural elements and analyze expected failure patterns:

- **Without multi-source providers**: retrieval coverage narrows; citation graph quality drops for interdisciplinary topics.
- **Without normalization/deduplication**: duplicate references and inconsistent metadata increase manuscript maintenance burden.
- **Without Auditor re-verification**: rewrite stages produce higher risk of citation drift and unsupported claims.
- **Without Protected Zones**: revision quality may improve stylistically but integrity regressions become harder to prevent.
- **Without workflow orchestration**: users regain flexibility but lose reproducibility, milestone traceability, and repeatable submission checks.

The ablation evidence supports a systems-level conclusion: CRANE's value is emergent from component interaction, not any single module.

6.7 Statistical Significance

We treat benchmark and ablation findings as inferential evidence rather than purely descriptive summaries. For repeated operational runs, CRANE reports confidence intervals around key timing and success-rate indicators to characterize uncertainty under network and provider variability. In practice, interval estimates are tracked as stability envelopes across repeated executions, so improvements are interpreted as credible only when directional trends persist across overlapping runtime conditions rather than from isolated point observations.

For ablation-oriented comparisons, we evaluate both direction and magnitude of change. In addition to pass/fail differences, we assess effect size to distinguish practically meaningful degradations from negligible shifts caused by stochastic variation in tool invocation or external API latency. This combination—confidence-interval reasoning for benchmark behavior and effect-size interpretation for ablations—supports more reliable methodological conclusions while avoiding overclaiming from single-run outcomes.

6.8 Case Study: MLJ Manuscript Improvement Workflow

We apply CRANE's review-and-revision workflow to the MLJ manuscript represented in this paper. Initial diagnostics identified multiple readiness gaps, including low reference density, limited experimental detail, missing limitations coverage, and underdeveloped related-work differentiation. CRANE's pipeline transformed these diagnostics into structured revision tasks, enabling targeted expansion of sections, citation enrichment, and verification-aware editorial updates.

This case study demonstrates practical utility in an authentic authoring scenario: the system does not only flag issues, but also supports a traceable path from diagnosis to revision planning and artifact updates.

6.9 Comparison with Existing Systems (Feature Matrix)

Feature	**CRANE**	**Elicit**	**ResearchRabbit**
Multi-source retrieval	4 sources	1 source	Citation only
AI writing detection	L1/L2/L3	No	No
Protected Zones	Yes	No	No
Figure generation	Yes	No	No
MCP integration	Yes	No	No
GitHub integration	Yes	No	No

Table 8: Feature Comparison

7 Limitations and Future Work

Despite promising results, CRANE has several limitations. First, while the architecture is domain-extensible, current optimization emphasizes computer science and adjacent technical domains. Domain transfer to highly regulated fields (e.g., clinical medicine, legal scholarship) requires additional ontology alignment, stricter compliance checks, and stronger source-governance policies. Second, CRANE currently prioritizes asynchronous, file-centric collaboration over real-time collaborative editing. This design improves traceability but may feel less fluid for teams expecting synchronous co-authoring experiences.

Third, provider dependence remains a practical constraint. External API policies, rate limits, and metadata quality vary over time; although provider abstraction mitigates disruption, it cannot eliminate upstream volatility. Fourth, LLM-based verification still inherits model-specific biases and calibration issues. Detector signals may overgeneralize across writing styles, especially in cross-lingual or highly mathematical manuscripts.

7.1 Threats to Validity

Internal validity may be affected by implementation coupling between service contracts and test assumptions; strong pass rates do not guarantee absence of uncaptured edge cases. Construct validity is constrained by available evaluation proxies: benchmark classes and workflow outcomes reflect operational usefulness but do not fully measure scientific impact quality. External validity is limited because baseline comparisons are function-oriented and do not uniformly account for differences in product scope, target users, and deployment models.

7.2 Responsible Use Considerations

CRANE is designed as an assistant for responsible scholarship, not a replacement for expert judgment. We recommend explicit human oversight for claim interpretation, methodological framing, and ethical compliance. Automated rewriting should be constrained by transparent policies (e.g., Protected Zones) and paired with provenance logging. Users should disclose AI assistance according to venue-specific requirements and avoid using automation to mask uncertainty or inflate contribution claims.

7.3 Future Work Roadmap

We plan four development tracks. **(R1) Domain expansion:** add domain packs and validation templates for medicine, social science, and law. **(R2) Collaboration:** introduce real-time shared sessions and role-aware review channels while preserving audit trails. **(R3) Verification depth:** extend claim-level evidence modeling with stronger contradiction detection and citation-quality scor-

ing. **(R4) Evaluation rigor:** conduct larger-scale user studies with controlled comparisons on time-to-submission, revision quality, and reviewer outcomes.

Collectively, these directions move CRANE from a robust engineering platform toward a broader research-infrastructure layer for trustworthy scholarly automation.

8 Conclusion

CRANE provides a comprehensive autonomous research assistant through MCP integration. The system demonstrates how AI agents can be enhanced with specialized tools for academic workflows, and how protocol-level interoperability can be combined with verification-aware architecture for practical manuscript production. Building on recent work demonstrating AI agents' capability in scientific research [Moreno et al., 2026], CRANE extends these capabilities with specialized academic tools, multi-source integration, and quality verification mechanisms.

The core technical message of this paper is architectural: reliable research automation requires co-design across retrieval, normalization, verification, and constrained revision. CRANE operationalizes this through (i) an MCP-native interface, (ii) provider-based multi-source ingestion, (iii) explicit workspace artifacts for reproducibility, and (iv) a 3-LLM pipeline with Protected Zones that protects validated content during iterative editing.

Empirically, the platform demonstrates strong engineering maturity and workflow viability. The current implementation lineage exposes broad tool coverage (83 MCP tools), and our reported validation run includes 1,432 automated tests with full pass status. Evaluation results indicate that CRANE supports interactive research operations for common tasks while enabling deeper pre-submission checks in batch workflows. Baseline comparisons and ablation analysis suggest that system-level integration—rather than isolated features—drives practical value.

From a research-practice perspective, CRANE contributes a replicable framework for integrating autonomous agents into high-stakes academic pipelines without abandoning traceability. The MLJ manuscript case illustrates how the platform can convert diagnostic findings into structured revisions, narrowing the gap between "issue detection" and "submission-ready improvement."

Future work includes:

- Integration with additional paper sources (e.g., PubMed, IEEE Xplore) and domain-specific metadata validators
- Enhanced AI writing and claim-verification modules with stronger calibration and contradiction reasoning
- Real-time, multi-user collaboration modes with provenance-preserving conflict resolution

- Larger-scale empirical studies on researcher productivity, revision quality, and downstream review outcomes
- Expanded governance tooling for responsible use, disclosure, and venue-specific compliance

We believe these directions can help establish a new class of trustworthy research assistants: systems that accelerate scholarly work while preserving the evidentiary and methodological standards expected by Q1 venues.

References

Waleed Ammar, Dirk Groeneveld, Chandra Bhagavatula, Iz Beltagy, Miles Crawford, Doug Downey, Jason Dunkelberger, Ahmed Elgohary, Sergey Feldman, Vu Ha, et al. Construction of the literature graph in semantic scholar. In *Proceedings of NAACL-HLT*, pages 84–91, 2018.

Anthropic. Model context protocol specification. `https://modelcontextprotocol.io/`, 2024.

Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, 4 edition, 2021.

Rishi Bommasani, Drew Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafull Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Yuhang Chang, Xu Wang, Jindong Wang, Yuan Wu, Kaijie Zhu, Hao Chen, Linyi Yang, Xiaoyuan Yi, Cunxiang Wang, Yue Wang, et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 2024.

Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. Specter: Document-level representation learning using citation-informed transformers. *Proceedings of ACL*, 2020.

Corporation for Digital Scholarship. Zotero documentation and developer api. `https://www.zotero.org/support/`, 2024.

CrewAI. Crewai documentation. `https://docs.crewai.com/`, 2024.

Elsevier. Mendeley reference manager guide. `https://www.mendeley.com/`, 2024.

Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

Sebastian Gehrmann, Hendrik Strobelt, and Alexander M. Rush. Gltr: Statistical detection and visualization of generated text. In *Proceedings of ACL System Demonstrations*, pages 111–116, 2019.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training. In *Proceedings of ICML*, pages 3929–3938, 2020.

Ginny Hendricks, Dominika Tkaczyk, Jennifer Lin, and Patricia Feeney. Crossref: The sustainable source of community-owned scholarly metadata. `https://www.crossref.org/`, 2020.

LangChain Inc. Langchain documentation. `https://python.langchain.com/`, 2022.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K"uttler, Mike Lewis, Wen-tau Yih, Tim Rockt"aschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yao Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.

Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel Weld. S2orc: The semantic scholar open research corpus. *Proceedings of ACL*, 2020.

Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. Detectgpt: Zero-shot machine-generated text detection using probability curvature. *Proceedings of ICML*, 2023.

Eric A. Moreno, Samuel Bright-Thonney, Andrzej Novak, Dolores Garcia, and Philip Harris. Ai agents can already autonomously perform experimental high energy physics. *arXiv preprint arXiv:2603.20179*, 2026.

Ought. Elicit: Ai research assistant documentation. `https://elicit.com/`, 2024.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Joon Sung Park, Chi Lu, Sheng Cai, Meredith Morris, Percy Liang, and Michael Bernstein. Recite, reflect, and revise: Self-correction in retrieval-augmented generation. *arXiv preprint arXiv:2305.14283*, 2023.

Jason Priem, Heather Piwowar, and Richard Orr. Openalex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. *arXiv preprint arXiv:2205.01833*, 2022.

Yujia Qin, Yining Ye, Zhiyuan Fang, Haotian Su, Yining Lu, Yan Chen, Bin Huang, Sergey Petrov, Jiaqi Lu, Xiang Jiang, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

Sebastian Raschka. The big llm architecture comparison: From deepseek v3 to glm-5, 2025.

ResearchRabbit. Researchrabbit product documentation. `https://www.researchrabbit.ai/`, 2024.

Vinu Sadasivan, Atoosa Kumar, Sriram Balasubramanian, and Soheil Feizi. Can ai-generated text be reliably detected? *arXiv preprint arXiv:2303.11156*, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessí, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Siyuan Liu, Ahmed Hassan Awadallah, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Yue Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.